Exercicios em C

Aluno:Ruan Silva Carvalho

Exercicio 01

#include <stdio.h> #include <math.h>

double aloha_puro(double g) { return g * exp(-2 * g); }

double aloha_slotted(double g) { return g * exp(-g); }

int main() { double g, sp, ss; double maxp = 0, maxs = 0, gp = 0, gs = 0; int i;

```
printf("  G     S(puro)   S(slotted)\n");

for (i = 1; i <= 20; i++) {
    g = i / 10.0;
    sp = aloha_puro(g);
    ss = aloha_slotted(g);
    printf("%.1f    %.4f    %.4f\n", g, sp, ss);

    // guarda o maior valor encontrado
    if (sp > maxp) {
        maxp = sp;
        gp = g;
    }
    if (ss > maxs) {
        maxs = ss;
        gs = g;
    }
}

printf("\nALOHA puro: maximo = %.4f em G = %.1f\n", maxp, gp);
printf("Slotted ALOHA: maximo = %.4f em G = %.1f\n", maxs, gs);

return 0;


}
```



Exercicio 02

#include <stdio.h>

// Lmin = taxa * 2 * Tp, onde Tp = distancia / velocidade double frame_minimo(double taxa, double distancia, double velocidade) { double tp = distancia / velocidade; return taxa * 2 * tp; }

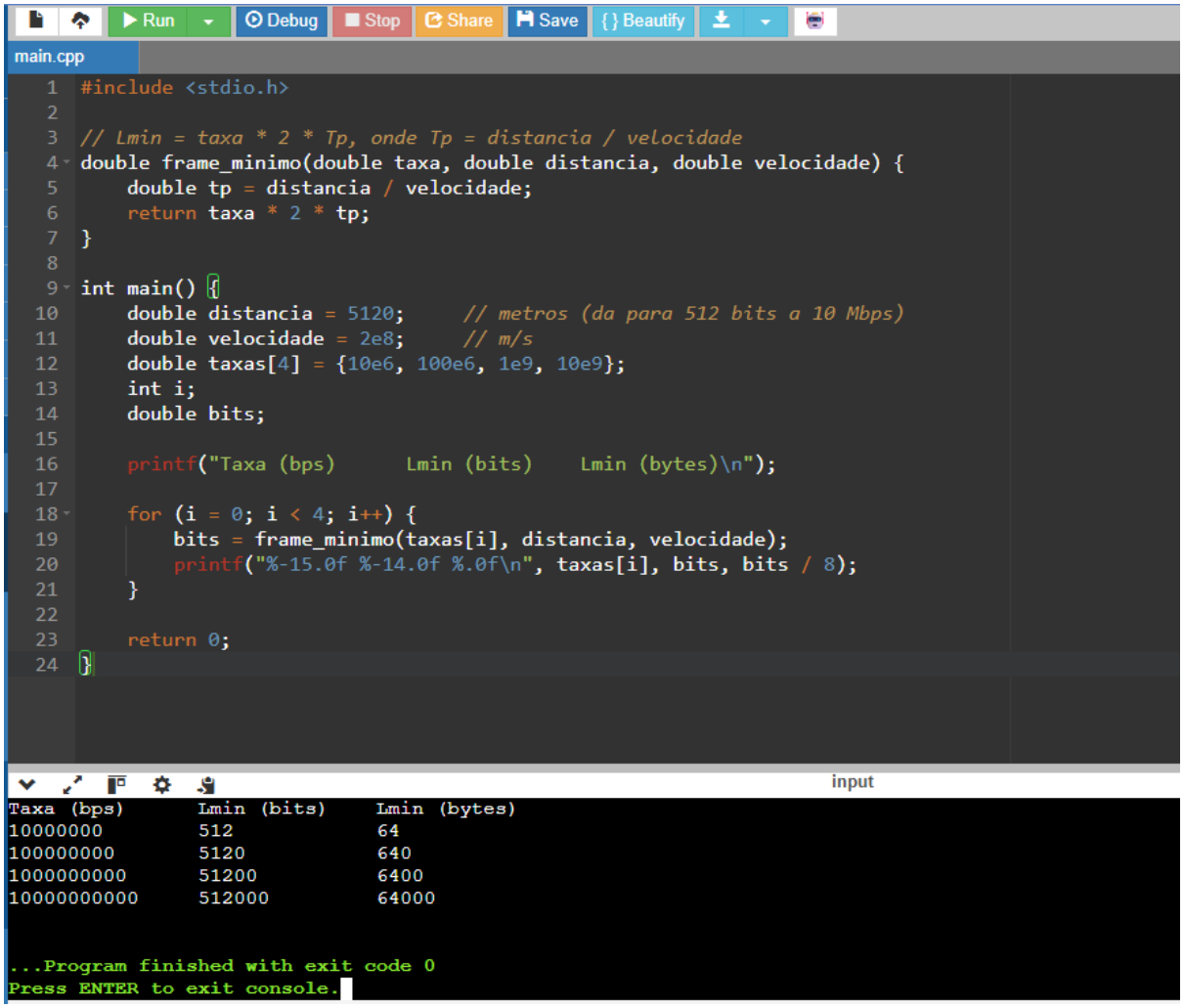
int main() { double distancia = 5120; // metros (da para 512 bits a 10 Mbps) double velocidade = 2e8; // m/s double taxas[4] = {10e6, 100e6, 1e9, 10e9}; int i; double bits;

```
printf("Taxa (bps)      Lmin (bits)    Lmin (bytes)\n");

for (i = 0; i < 4; i++) {
    bits = frame_minimo(taxas[i], distancia, velocidade);
    printf("%-15.0f %-14.0f %.0f\n", taxas[i], bits, bits / 8);
}

return 0;
```

}



Exercicio 03

#include <stdio.h>

// calcula t3, t4 e os bits enviados por A e por C void colisao(double taxa, double dist, double vel, double t1, double t2, double *t3, double *t4, double *bitsA, double *bitsC) { double tp = dist / vel;

```
*t3 = t1 + tp;              // C escuta o primeiro bit de A
*t4 = t2 + tp;              // A escuta o primeiro bit de C
*bitsA = (*t4 - t1) * taxa;
*bitsC = (*t3 - t2) * taxa;
```

}

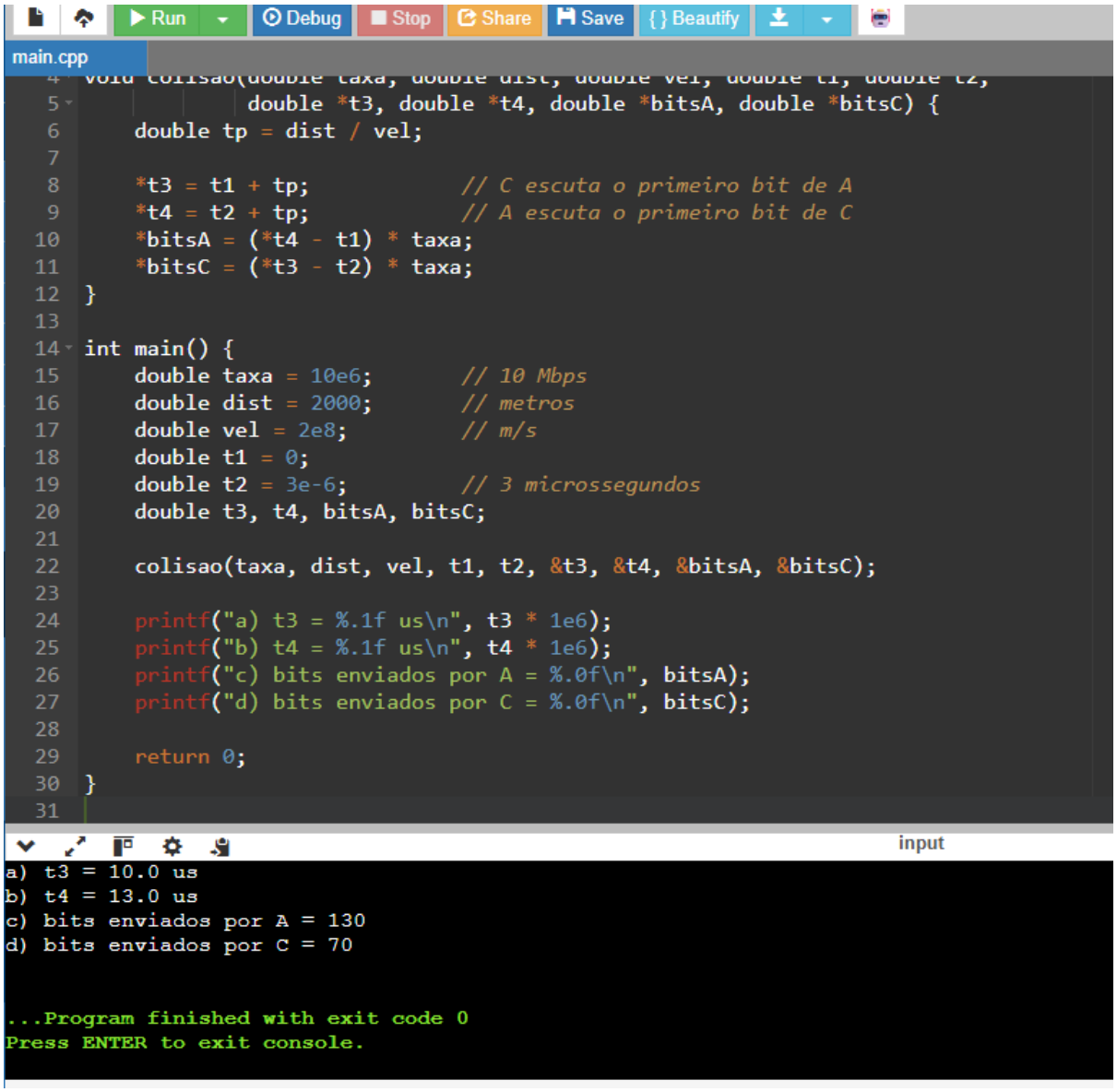
int main() { double taxa = 10e6; // 10 Mbps double dist = 2000; // metros double vel = 2e8; // m/s double t1 = 0; double t2 = 3e-6; // 3 microssegundos double t3, t4, bitsA, bitsC;

```
colisao(taxa, dist, vel, t1, t2, &t3, &t4, &bitsA, &bitsC);

printf("a) t3 = %.1f us\n", t3 * 1e6);
printf("b) t4 = %.1f us\n", t4 * 1e6);
printf("c) bits enviados por A = %.0f\n", bitsA);
printf("d) bits enviados por C = %.0f\n", bitsC);

return 0;
```

}



Exercicio 04

```
#include <stdio.h>


// (a) sem limite: a estacao manda todos os frames de uma vez no
poll

int polling_sem_limite(int estacoes, int frames, int dados, int
ctrl) {

    int total = 0;

    int i;


    for (i = 0; i < estacoes; i++) {

        total = total + ctrl;                  // poll

        total = total + frames * (dados + ctrl);  // frames + ACKs

    }

    return total;

}


// (b) um frame por poll: cada frame precisa de um poll

int polling_um_frame(int estacoes, int frames, int dados, int ctrl)
{

    int total = 0;

    int i, j;


    for (i = 0; i < estacoes; i++) {

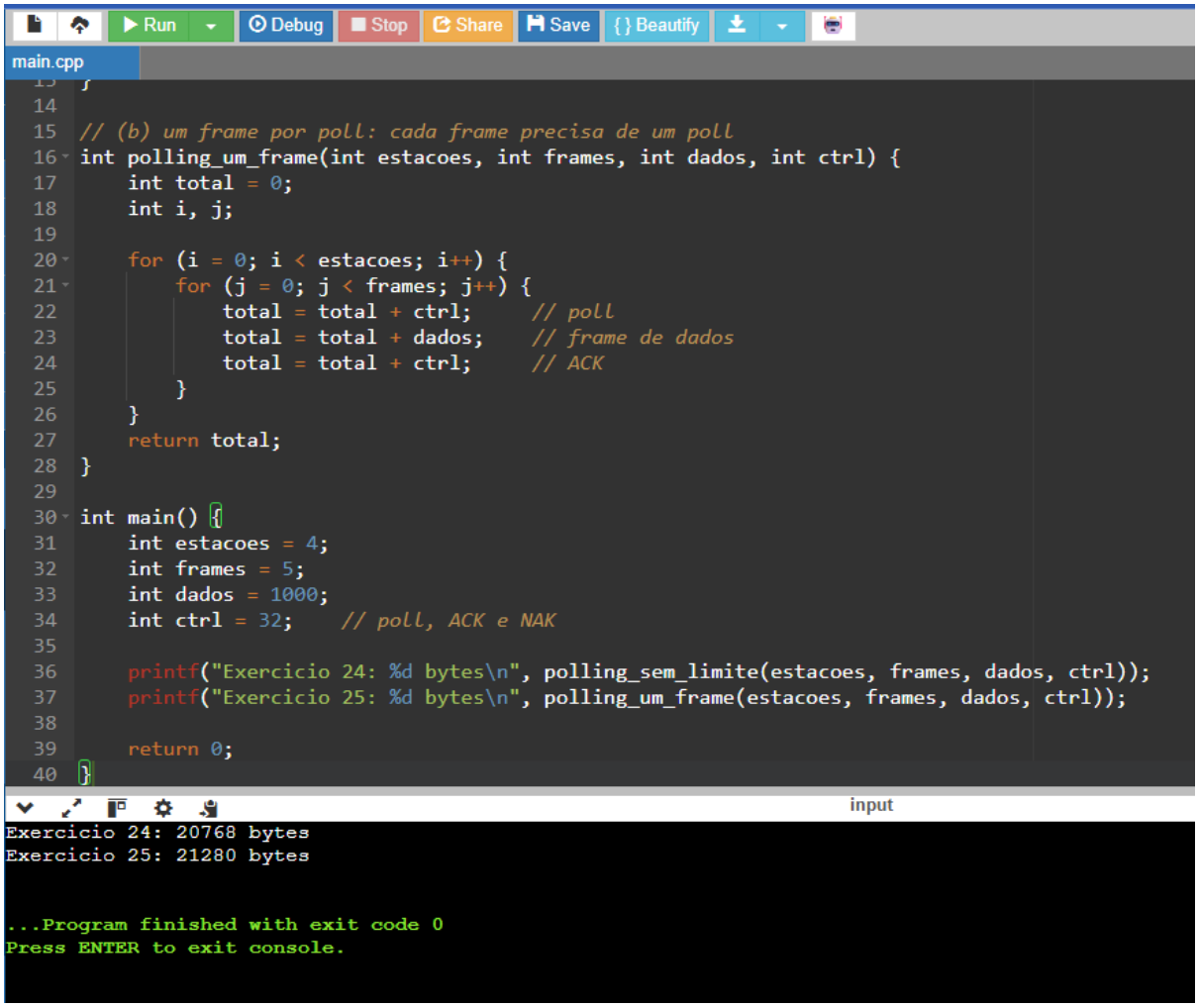
        for (j = 0; j < frames; j++) {
```

```
            total = total + ctrl;     // poll
            total = total + dados;    // frame de dados
            total = total + ctrl;     // ACK
        }
    }
    return total;
}

int main() {
    int estacoes = 4;
    int frames = 5;
    int dados = 1000;
    int ctrl = 32;    // poll, ACK e NAK

    printf("Exercicio 24: %d bytes\n", polling_sem_limite(estacoes,
frames, dados, ctrl));
    printf("Exercicio 25: %d bytes\n", polling_um_frame(estacoes,
frames, dados, ctrl));

    return 0;
}
```



Exercicio 05

```
#include <stdio.h>


#define N 8


int w[N][N];


// gera a tabela de Walsh de tamanho n
void gera_walsh(int n) {
   int k, i, j;


   w[0][0] = 1;
```

```
    for (k = 1; k < n; k = k * 2) {
        for (i = 0; i < k; i++) {
            for (j = 0; j < k; j++) {
                w[i][j + k] = w[i][j];
                w[i + k][j] = w[i][j];
                w[i + k][j + k] = -w[i][j];   // complemento
            }
        }
    }
}

// produto interno de duas sequencias
int produto(int a[], int b[]) {
    int k, soma = 0;
    for (k = 0; k < N; k++) {
        soma = soma + a[k] * b[k];
    }
    return soma;
}

int main() {
    int i, j, k;
    int ok = 1;
    int dados[N] = {1, -1, 0, 1, -1, 1, 0, -1};   // -1 = bit 0, 1 = bit 1, 0 = silencio
    int canal[N] = {0};

    gera_walsh(N);
```

```
printf("Tabela W8:\n");
for (i = 0; i < N; i++) {
   for (j = 0; j < N; j++) {
      printf("%2d ", w[i][j]);
   }
   printf("\n");
}

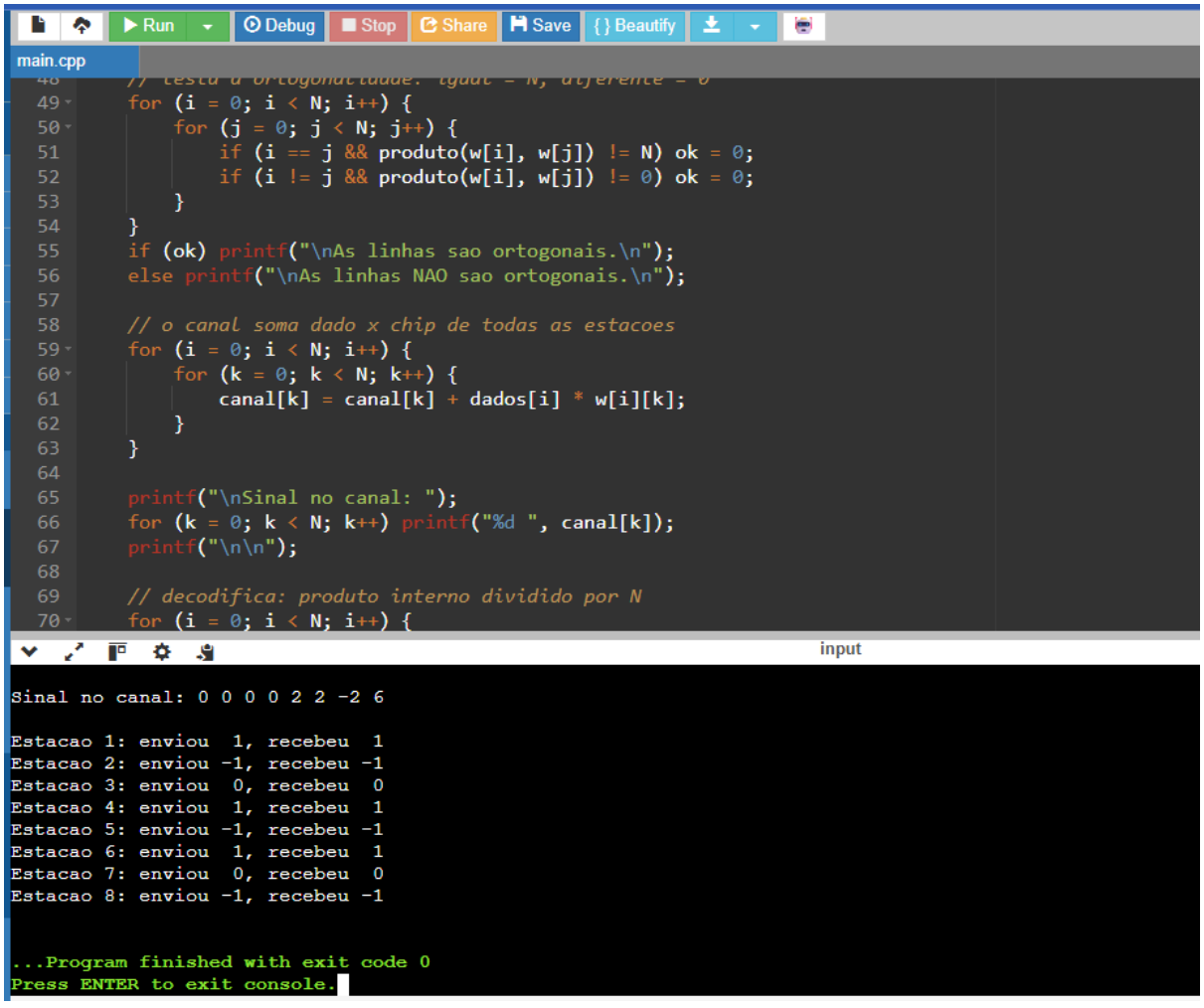
// testa a ortogonalidade: igual = N, diferente = 0
for (i = 0; i < N; i++) {
   for (j = 0; j < N; j++) {
      if (i == j && produto(w[i], w[j]) != N) ok = 0;
      if (i != j && produto(w[i], w[j]) != 0) ok = 0;
   }
}
if (ok) printf("\nAs linhas sao ortogonais.\n");
else printf("\nAs linhas NAO sao ortogonais.\n");

// o canal soma dado x chip de todas as estacoes
for (i = 0; i < N; i++) {
   for (k = 0; k < N; k++) {
      canal[k] = canal[k] + dados[i] * w[i][k];
   }
}

printf("\nSinal no canal: ");
for (k = 0; k < N; k++) printf("%d ", canal[k]);
printf("\n\n");
```

```
    // decodifica: produto interno dividido por N
    for (i = 0; i < N; i++) {
        printf("Estacao %d: enviou %2d, recebeu %2d\n", i + 1, dados[i], produto(canal, w[i]) / N);
    }

    return 0;
}
```



Exercício 1: ALOHA puro e slotted

Nesse exercício eu tinha que calcular o throughput (S) do ALOHA puro e do slotted ALOHA para vários valores de G e ver onde cada um chega no máximo.

Eu criei duas funções, uma para cada fórmula: `aloha_puro`, que faz G·e^(−2G), e `aloha_slotted`, que faz G·e^(−G). No `main` usei um `for` de 1 até 20 e dividi o contador por 10 para gerar o G de 0,1 até 2,0. Fiz assim, e não somando 0,1 a cada volta, porque números `double` vão acumulando erro de arredondamento.

A cada volta o programa imprime a linha da tabela e compara o valor com o maior que já tinha achado. Se for maior, guarda o valor e o G dele. No final mostra os dois máximos.

Deu 0,1839 para o ALOHA puro em G = 0,5 e 0,3679 para o slotted em G = 1,0, o que bate com os 18,4% e 36,8% dos slides. Dá para ver que o slotted é o dobro do puro, porque o tempo em que o frame pode colidir cai pela metade.

Exercício 2: Frame mínimo no CSMA/CD

Aqui o objetivo era calcular o menor tamanho de frame para que a estação ainda esteja transmitindo quando perceber a colisão.

A função `frame_minimo` primeiro calcula o tempo de propagação (Tp = distância ÷ velocidade) e depois multiplica taxa × 2 × Tp. Multiplica por 2 porque o sinal precisa ir até a outra ponta e a notícia da colisão precisa voltar.

No `main` usei a velocidade de 2×10^8 m/s e a distância de 5120 m. Escolhi esse valor porque é o que dá 512 bits para 10 Mbps, igual ao exemplo dos slides. Depois fiz um `for` pelas taxas de 10 Mbps, 100 Mbps, 1 Gbps e 10 Gbps, imprimindo em bits e em bytes.

Os resultados foram 512, 5.120, 51.200 e 512.000 bits. Percebi que o frame mínimo cresce junto com a taxa, então quanto mais rápida a rede, maior o frame ou menor a distância precisa ser.

Exercício 3: Instantes da colisão

Esse é o Exercício 17 do livro. A estação A começa a transmitir em t1 e a C em t2, e eu tinha que achar quando cada uma percebe a colisão e quantos bits cada uma já tinha enviado.

Fiz uma função `colisao` que devolve os quatro resultados usando ponteiros, já que uma função em C só retorna um valor. As contas são:

- `t3 = t1 + Tp`, que é quando o primeiro bit de A chega em C;
- `t4 = t2 + Tp`, que é quando o primeiro bit de C chega em A;
- bits de A = (t4 − t1) × taxa, porque A transmitiu de t1 até t4;

- bits de C = (t3 − t2) × taxa, porque C transmitiu de t2 até t3.

O programa supõe que t1 ≤ t2 e que a diferença entre eles é menor que Tp, senão nem teria colisão.

Com 10 Mbps, 2000 m e t2 = 3 µs, deu t3 = 10 µs, t4 = 13 µs, 130 bits para A e 70 bits para C. Como a função é genérica, dá para usar no Exercício 18 só trocando os números no `main`.

Exercício 4: Polling

Aqui eu tinha que contar quantos bytes passam na rede no polling, com 4 estações secundárias, cada uma com 5 frames de 1000 bytes para enviar. Poll, ACK e NAK têm 32 bytes.

Fiz duas funções. Na `polling_sem_limite` (Exercício 24), cada estação recebe um poll só e manda os 5 frames de uma vez, então por estação são 32 bytes de poll mais 5 × (1000 + 32). Na `polling_um_frame` (Exercício 25), cada estação só pode mandar um frame por poll, então cada frame custa poll + dados + ACK, e isso é repetido 5 vezes por estação. Usei `for` nas duas para somar tudo em `total`.

Deu 20.768 bytes no primeiro caso e 21.280 bytes no segundo. A diferença é de 512 bytes, que são os 16 polls a mais (20 contra 4) vezes 32 bytes.

O enunciado não explica como contar, então eu considerei um ACK para cada frame de dados e não contei uma rodada final de NAK. Se o professor contar diferente, o resultado pode mudar um pouco.

Exercício 5: Walsh e CDMA

Esse foi o mais comprido. Tinha que gerar a tabela de Walsh para 8 estações e simular o CDMA.

A função `gera_walsh` começa com W1 = [1] e vai dobrando o tamanho da tabela. Em cada etapa copia o bloco para a direita e para baixo e coloca o complemento (sinal trocado) no canto de baixo à direita. Fiz isso com três `for` aninhados.

A função `produto` faz o produto interno de duas linhas, multiplicando posição por posição e somando.

No `main` primeiro imprimo a tabela W8 e testo a ortogonalidade: o produto de uma linha com ela mesma tem que dar 8, e com qualquer outra linha tem que dar 0. Deu certo.

Depois cada estação tem um dado, que pode ser 1 (bit 1), −1 (bit 0) ou 0 (não está enviando). Eu multipliquei o dado de cada estação pela linha dela e somei tudo no vetor `canal`, que é o que passa de fato no meio de transmissão.

Para decodificar, faço o produto interno do `canal` com a linha da estação e divido por 8. Isso funciona porque as outras estações dão 0 por causa da ortogonalidade, e só sobra a da estação que eu quero.

O canal ficou `0 0 0 0 2 2 -2 6`, e as 8 estações recuperaram exatamente o que enviaram, inclusive as duas que estavam em silêncio.